PROMPT MESTRE

Engenharia Reversa, Documentação Completa e Plano de Remake do Rising Lands

Documento consolidado - 70 seções técnicas

Rising Lands • Preservação digital • Engenharia reversa • Reimplementação

Sobre este documento

Este PDF consolida o prompt completo para orientar uma IA/agente técnico na análise integral dos arquivos do Rising Lands, na documentação do funcionamento do jogo e na criação progressiva de um remake moderno e compatível com Windows 10/11. **O texto deve ser utilizado como instrução de projeto e preservado integralmente.**

Estrutura geral

- Seções 1-10: objetivos, método, executáveis, game loop e timing
- Seções 11-24: mapas, unidades, economia, IA, eventos e missões
- Seções 25-36: gráficos, áudio, interface, saves, memória e especificação técnica
- Seções 37-48: arquitetura e desenvolvimento do remake
- Seções 49-64: rastreabilidade, dados, compatibilidade e preservação
- Seções 65-70: ordem de execução, entregas e objetivo final

PROMPT MESTRE — ENGENHARIA REVERSA, DOCUMENTAÇÃO COMPLETA E PLANO DE REMAKE DO RISING LANDS

Você tem autorização para ler, copiar para uma área de trabalho e analisar todos os arquivos da pasta do Rising Lands.

REGRAS DE SEGURANÇA

- Não altere os arquivos originais.
- Não exclua arquivos originais.
- Não sobrescreva arquivos originais.
- Não modifique saves originais.
- Não modifique executáveis originais.
- Não execute operações destrutivas.
- Trabalhe sempre em cópias.
- Crie uma área de trabalho separada.
- Registre todas as alterações feitas em cópias.
- Preserve evidências de como cada conclusão foi obtida.
- Pode criar scripts, ferramentas, relatórios, conversores, protótipos e código do remake em diretórios separados.

---

1. PAPEL QUE VOCÊ DEVE ASSUMIR

Quero que você atue como uma equipe técnica multidisciplinar especializada em:

- engenharia reversa de jogos antigos;
- análise de binários PE32;
- reverse engineering de formatos proprietários;
- análise de arquivos binários;
- estruturas de dados;
- jogos 2D de estratégia em tempo real;
- engines antigas baseadas em DirectX 3/5;
- arquitetura de software;
- desenvolvimento de engines modernas;
- preservação de software;
- criação de ferramentas de conversão;
- migração de jogos antigos para Windows 10/11;
- documentação técnica;
- análise de saves;
- análise de mapas;
- análise de eventos;
- sistemas de inteligência artificial;
- pathfinding;
- sistemas de triggers;
- gráficos 2D;
- sprites;
- animações;
- áudio;
- game design;
- reconstrução de lógica de gameplay.

Seu objetivo é analisar **todos os arquivos disponíveis do jogo Rising Lands** e produzir documentação técnica suficiente para permitir a reconstrução completa do jogo em uma nova engine moderna, nativa para Windows 10/11, preservando ao máximo:

- gameplay;
- regras;
- campanha;

- missões;
- unidades;
- construções;
- pesquisas;
- mapas;
- gráficos;
- sons;
- IA;
- eventos;
- comportamento;
- timing;
- balanceamento;
- interface;
- sistema de save/load.

---

2. OBJETIVO CENTRAL

O objetivo final é conseguir responder, com documentação, evidências e ferramentas:

> *Como o Rising Lands funciona internamente e como podemos reconstruí-lo do zero de maneira fiel, moderna, estável e compatível com Windows 10/11?*

Não trate isso como uma análise superficial.

Não faça apenas uma listagem de arquivos.

Não produza apenas hipóteses.

Quero reconstruir o funcionamento técnico e lógico do jogo.

---

3. PRINCÍPIO DE ANÁLISE

Todo arquivo disponível deve ser considerado potencialmente importante.

Não ignore arquivos apenas porque:

- a extensão é desconhecida;
- parecem pequenos;
- parecem antigos;
- parecem binários;
- parecem redundantes;
- não possuem documentação.

Sempre diferencie claramente:

- **CONFIRMADO**
- **FORTE EVIDÊNCIA**
- **PROVÁVEL**
- **HIPÓTESE**
- **DESCONHECIDO**

Nunca apresente uma hipótese como fato.

Para cada descoberta importante, explique:

- arquivo de origem;
- offset, endereço ou região;
- evidência;
- método usado;
- conclusão;
- grau de certeza.

---

4. ESTRUTURA DE TRABALHO

Crie uma estrutura semelhante a:

```
RisingLandsResearch/
├── original/
├── working/
├── extracted/
├── converted/
├── documentation/
├── tools/
├── experiments/
├── reverse_engineering/
├── source_remake/
├── tests/
├── logs/
└── reports/
```

A pasta `original/` deve ser tratada como somente leitura.

---

5. INVENTÁRIO COMPLETO

Comece fazendo um inventário recursivo de todos os arquivos do jogo.

Para cada arquivo registre:

- nome;
- caminho;
- extensão;
- tamanho;
- SHA-256;
- magic bytes;
- assinatura;
- formato provável;
- conteúdo textual ou binário;
- nível de entropia;
- existência de compressão;
- strings relevantes;
- relação aparente com outros arquivos;
- importância provável;
- necessidade de análise posterior.

Crie:

```
documentation/file_inventory.md
documentation/file_inventory.csv
```

Classifique os arquivos em categorias:

- executáveis;
- DLLs;
- configurações;
- gráficos;
- sprites;
- tiles;
- mapas;
- sons;
- músicas;
- animações;
- fontes;
- textos;
- regras;
- eventos;

- IA;
- unidades;
- construções;
- tecnologias;
- saves;
- instaladores;
- arquivos desconhecidos.

---

6. IDENTIFICAÇÃO DA ARQUITETURA DO JOGO

Monte progressivamente uma visão estrutural semelhante a:

```
Rising Lands
├── Engine
│   ├── Main Loop
│   ├── Rendering
│   ├── Input
│   ├── Audio
│   ├── Timing
│   ├── Memory
│   ├── File I/O
│   ├── Save/Load
│   └── Networking
│
├── World
│   ├── Map
│   ├── Terrain
│   ├── Tiles
│   ├── Objects
│   ├── Resources
│   └── Environment
│
├── Gameplay
│   ├── Units
│   ├── Buildings
│   ├── Economy
│   ├── Research
│   ├── Combat
│   ├── Trade
│   └── Alliances
│
├── Simulation
│   ├── AI
│   ├── Pathfinding
│   ├── Events
│   ├── Missions
│   ├── Spawn System
│   └── Triggers
│
└── Presentation
    ├── Graphics
    ├── UI
    ├── Animation
    ├── Audio
    └── Text
```

Atualize esse mapa conforme novas descobertas surgirem.

Crie:

```
documentation/architecture_overview.md
```

---

7. ANÁLISE DOS EXECUTÁVEIS

Analise todos os executáveis relacionados ao jogo.

Especialmente:

```
RISING.EXE
```

e quaisquer versões:

- fixed;
- patch;
- alternate build;
- CD version;
- modified version.

Para cada executável, identifique:

- arquitetura;
- PE32/PE32+;
- timestamp;
- compilador provável;
- linker provável;
- subsistema;
- entry point;
- imports;
- exports;
- seções;
- tamanho;
- strings;
- recursos embutidos;
- dependências;
- referências a arquivos;
- APIs gráficas;
- APIs de áudio;
- APIs de input;
- APIs de arquivos;
- APIs de temporização;
- APIs de rede.

Procure especialmente:

```
DirectDraw
DirectSound
DirectPlay
WINMM
GetTickCount
timeGetTime
Sleep
CreateFile
ReadFile
WriteFile
fopen
fclose
fread
fwrite
malloc
calloc
free
memcpy
memset
```

Crie:

```
documentation/executable_map.md
```

Use tabela semelhante a:

```
| Endereço | Função provável | Evidência | Confiança |
|---|---|---|---|
| 0x004xxxxx | Game loop | chamadas + timing | CONFIRMADO |
| 0x004xxxxx | Load map | strings + leitura | PROVÁVEL |
| 0x004xxxxx | Unit update | acesso à tabela de unidades | FORTE EVIDÊNCIA |
```

---

8. COMPARAÇÃO ENTRE EXECUTÁVEIS

Se existirem múltiplas versões de `RISING.EXE`:

- faça diff binário;
- diff de código;
- compare imports;
- compare seções;
- compare strings;
- compare funções;
- identifique mudanças;
- identifique patches;
- identifique correções de timing;
- identifique correções de compatibilidade;
- identifique bugs introduzidos.

Crie:

```
documentation/exe_diff.md
```

Documente:

- endereço;
- bytes originais;
- bytes alterados;
- função afetada;
- efeito observado;
- hipótese;
- risco.

---

9. GAME LOOP

Descubra como funciona o loop principal.

Identifique:

- update;
- render;
- input;
- IA;
- física;
- pathfinding;
- eventos;
- som;
- timers;
- frame pacing.

Determine se o jogo usa:

- loop contínuo;
- event loop;
- busy wait;
- GetTickCount;
- timeGetTime;
- Sleep;

- polling.

Crie:

```
documentation/game_loop.md
```

---

10. TIMING E VELOCIDADE

Descubra como o jogo controla:

- FPS;
- velocidade geral;
- velocidade das unidades;
- produção;
- pesquisa;
- animação;
- combate;
- timers;
- eventos;
- spawn;
- IA.

Determine se a lógica depende de:

- CPU;
- clock;
- FPS;
- ciclos;
- ticks;
- delays artificiais.

Esse ponto é crítico porque o jogo original pode ficar rápido demais em CPUs modernas.

Documente:

```
documentation/timing.md
```

Inclua fórmulas e pseudocódigo.

---

11. MAPAS

Identifique todos os arquivos relacionados a mapas.

Descubra:

- largura;
- altura;
- tamanho do tile;
- tiles;
- camadas;
- terreno;
- altura;
- passabilidade;
- colisão;
- objetos;
- recursos;
- spawn points;
- construções iniciais;
- unidades iniciais;
- eventos;
- scripts.

Crie:

```
documentation/maps.md
```

Desenvolva um parser.

Converta mapas para um formato intermediário moderno:

```
{
  "width": 0,
  "height": 0,
  "tileSize": 0,
  "tiles": [],
  "objects": [],
  "resources": [],
  "units": [],
  "buildings": [],
  "spawns": [],
  "events": []
}
```

Se possível, crie um visualizador de mapa.

---

12. TILES E TERRENO

Descubra:

- tile IDs;
- transições;
- água;
- terra;
- obstáculos;
- terreno caminhável;
- terreno construível;
- animações;
- overlays.

Documente:

```
documentation/terrain.md
```

---

13. UNIDADES

Mapeie toda a estrutura das unidades.

Para cada unidade tente identificar:

- ID;
- nome;
- classe;
- facção;
- sprite;
- animações;
- direção;
- HP;
- dano;
- defesa;
- velocidade;
- alcance;
- cadência;
- custo;
- requisitos;
- transporte;
- carga;
- recurso carregado;

- IA;
- estados;
- alvo;
- proprietário;
- posição;
- path;
- tecnologia relacionada.

Crie:

```
documentation/units.md
```

Se possível, exporte para:

```
{
  "id": 1,
  "name": "Example",
  "hp": 100,
  "speed": 2,
  "damage": 10,
  "range": 1
}
```

---

14. ESTADOS DAS UNIDADES

Descubra estados como:

- idle;
- walking;
- attacking;
- gathering;
- carrying;
- depositing;
- building;
- dying;
- casting;
- entering building;
- leaving building.

Documente IDs internos e transições.

Crie:

```
documentation/unit_states.md
```

---

15. CONSTRUÇÕES

Mapeie todas as construções.

Identifique:

- ID;
- nome;
- sprite;
- footprint;
- tamanho;
- HP;
- custo;
- capacidade;
- construção;
- tempo;
- produção;
- armazenamento;

- pesquisa;
- unidades treinadas;
- tecnologias;
- efeitos especiais.

Crie:

```
documentation/buildings.md
```

---

16. RECURSOS

Identifique todos os recursos.

Descubra:

- tipos;
- IDs;
- coleta;
- armazenamento;
- transporte;
- consumo;
- produção;
- limite;
- interface;
- relação com trabalhadores;
- relação com construções.

Investigue:

- memória;
- saves;
- arquivos de regras.

Documente offsets conhecidos.

Crie:

```
documentation/resources.md
```

---

17. ECONOMIA

Documente:

- coleta;
- depósito;
- custos;
- produção;
- geração;
- consumo;
- população;
- capacidade;
- armazenamento;
- trade.

Crie:

```
documentation/economy.md
```

---

18. PESQUISA

Mapeie todas as tecnologias.

Para cada tecnologia:

- ID;
- nome;
- categoria;
- pré-requisitos;
- custo;
- tempo;
- efeito;
- modificadores;
- unidades liberadas;
- construções liberadas.

Crie:

```
documentation/research.md
```

---

19. COMBATE

Descubra:

- fórmula de dano;
- defesa;
- armadura;
- alcance;
- projéteis;
- splash;
- velocidade do projétil;
- cooldown;
- precisão;
- alvo;
- morte;
- efeitos especiais.

Crie:

```
documentation/combat.md
```

---

20. IA

Investigue profundamente a IA.

Descubra:

- estados;
- prioridades;
- coleta;
- construção;
- recrutamento;
- defesa;
- ataque;
- expansão;
- pesquisa;
- alianças;
- comércio;
- reação ao jogador;
- timers;
- triggers;
- comportamento por missão.

Crie:

```
documentation/ai.md
```

Se a IA estiver hardcoded no EXE, documente as funções.

---

21. PATHFINDING

Descubra:

- grid;
- algoritmo;
- custo;
- bloqueios;
- recalculação;
- path cache;
- movimento em grupo;
- colisão;
- desvio.

Crie:

```
documentation/pathfinding.md
```

---

22. EVENTOS E TRIGGERS

Esse é um dos sistemas mais importantes.

Analise:

```
EVENTS
EVENTS.INI
WORLD
arquivos de missão
scripts
```

Descubra:

- triggers;
- timers;
- condições;
- flags;
- spawn;
- mensagens;
- objetivo;
- vitória;
- derrota;
- diálogo;
- cutscene;
- mudança de estado;
- criação de unidades;
- remoção de unidades;
- mudanças de mapa.

Crie:

```
documentation/events.md
```

---

23. MISSÕES

Para cada missão, crie um documento individual.

Exemplo:

```
documentation/missions/mission_01.md
documentation/missions/mission_02.md
```

Formato:

```
# Missão X

## Estado inicial

- recursos
- unidades
- construções
- jogadores
- mapa

## Eventos

00:00 -> ...
02:30 -> ...
Trigger X -> ...
Trigger Y -> ...

## Objetivos

...

## Condição de vitória

...

## Condição de derrota

...

## Spawns

...

## IA

...
```

---

24. SPAWN DE UNIDADES

Mapeie especificamente:

- quem cria;
- quando;
- onde;
- quantidade;
- tipo;
- proprietário;
- alvo inicial;
- IA inicial;
- trigger.

Crie:

```
documentation/spawn_system.md
```

---

25. GRÁFICOS

Analise todos os arquivos gráficos.

Descubra:

- formato;
- cabeçalho;
- largura;
- altura;
- profundidade;
- paleta;
- transparência;
- compressão;
- frames;
- animações;
- direções;
- sprites;
- tiles;
- UI;
- efeitos.

Crie conversores.

Converta para:

```
PNG
sprite sheets
texture atlases
```

Crie:

```
documentation/graphics.md
```

---

26. ANIMAÇÕES

Identifique:

- frames;
- duração;
- direção;
- sequência;
- loop;
- eventos de frame.

Crie:

```
documentation/animations.md
```

---

27. ÁUDIO

Analise:

```
SOUND
WAV
MIDI
CD Audio
outros formatos
```

Identifique:

- música;
- efeitos;
- vozes;
- loops;
- volume;
- triggers;
- IDs.

Converta quando necessário para:

```
WAV
OGG
```

Crie:

```
documentation/audio.md
```

---

28. INTERFACE

Reconstrua:

- HUD;
- botões;
- menus;
- cursores;
- ícones;
- tooltips;
- barras;
- minimapa;
- seleção;
- mensagens.

Documente:

- posição;
- tamanho;
- sprite;
- função.

Crie:

```
documentation/ui.md
```

---

29. INPUT

Documente:

- mouse;
- clique;
- duplo clique;
- seleção;
- drag;
- atalhos;
- teclado;
- câmera;
- zoom;
- velocidade;
- pausa.

Crie:

```
documentation/input.md
```

---

30. SAVEGAME

Faça engenharia reversa completa de `.sav`.

Descubra:

- assinatura;
- versão;
- missão;
- nome;

- timestamp;
- recursos;
- unidades;
- construções;
- mapa;
- pesquisas;
- eventos;
- IA;
- RNG;
- flags;
- checksum;
- duplicações;
- estruturas internas.

Crie:

```
documentation/save_format.md
```

Crie um parser:

```
tools/save_parser
```

Saída ideal:

```
{
  "mission": 1,
  "resources": {
    "resource1": 999,
    "resource2": 999,
    "resource3": 999
  },
  "units": [],
  "buildings": [],
  "research": [],
  "events": []
}
```

---

31. MEMÓRIA EM RUNTIME

Se necessário, execute o jogo e monitore memória.

Mapeie:

- tabela de unidades;
- tabela de construções;
- recursos;
- mapa;
- eventos;
- timers;
- IA;
- jogadores;
- objetos.

Relacione:

```
arquivo -> memória -> comportamento
```

Crie:

```
documentation/memory_map.md
```

Quando possível, descreva estruturas como:

```
struct Unit {
    int id;
    int owner;
    int x;
    int y;
    int hp;
```

```
    int target;
    int state;
};
```

Mesmo que inicialmente sejam incompletas.

---

32. CRASHES

Documente todos os crashes observados.

Para cada crash:

- endereço;
- tipo;
- instrução;
- registradores;
- memória acessada;
- ação do jogador;
- estado do jogo;
- função provável;
- causa provável;
- correção;
- versão afetada.

Crie:

```
documentation/crashes.md
```

---

33. LOG DE ENGENHARIA REVERSA

Mantenha:

```
documentation/research_log.md
```

Formato:

```
Data:
Arquivo:
Ferramenta:
Descoberta:
Evidência:
Confiança:
Hipótese:
Próximo passo:
```

---

34. STRINGS

Extraia strings de:

- EXE;
- DLLs;
- arquivos binários;
- saves;
- mapas;
- eventos.

Classifique por:

- arquivos;
- erros;
- UI;
- missões;
- nomes;

- debug;
- engine;
- recursos.

Crie:

```
documentation/strings.md
```

---

35. FORMATOS DESCONHECIDOS

Se encontrar um formato desconhecido:

1. examine magic bytes;
2. compare arquivos semelhantes;
3. extraia strings;
4. examine entropia;
5. identifique padrões;
6. busque cabeçalhos;
7. teste hipóteses;
8. compare com memória;
9. compare com EXE;
10. crie parser experimental.

Nunca abandone um formato sem registrar o que já foi descoberto.

---

36. ESPECIFICAÇÃO TÉCNICA DO JOGO

Ao final da análise, crie:

```
documentation/game_specification.md
```

Esse documento deve ser a "bíblia técnica" do Rising Lands.

Deve conter:

- arquitetura;
- game loop;
- timing;
- memória;
- mapas;
- terreno;
- unidades;
- construções;
- economia;
- recursos;
- pesquisa;
- combate;
- IA;
- pathfinding;
- eventos;
- missões;
- gráficos;
- animações;
- áudio;
- interface;
- input;
- saves;
- regras;
- fórmulas.

---

37. OBJETIVO DO REMAKE

Depois da análise, planeje um remake moderno.

Objetivos:

- Windows 10;
- Windows 11;
- 64-bit;
- hardware moderno;
- estabilidade;
- resolução moderna;
- modo janela;
- fullscreen;
- timing correto;
- suporte a assets originais;
- manutenção futura.

---

38. TECNOLOGIA RECOMENDADA

Preferência inicial:

```
C# + MonoGame
```

Mas avalie se outra tecnologia seria melhor.

Possibilidades:

- C# + MonoGame;
- C++ + SDL3;
- Godot;
- custom engine.

Justifique a escolha.

---

39. ARQUITETURA DO REMAKE

Estrutura sugerida:

```
RisingLandsRemake/
├── Engine/
│   ├── Core/
│   ├── Rendering/
│   ├── Audio/
│   ├── Input/
│   ├── Timing/
│   ├── Resources/
│   └── Serialization/
│
├── Game/
│   ├── World/
│   ├── Units/
│   ├── Buildings/
│   ├── Economy/
│   ├── Research/
│   ├── Combat/
│   ├── AI/
│   ├── Pathfinding/
│   └── Missions/
│
├── Data/
│   ├── Maps/
│   ├── Units/
```

```
│   ├── Buildings/
│   ├── Research/
│   ├── Missions/
│   └── Audio/
│
├── Importers/
│   ├── Graphics/
│   ├── Audio/
│   ├── Maps/
│   ├── Saves/
│   └── Events/
│
├── Tools/
└── Tests/
```

---

40. DESENVOLVIMENTO ORIENTADO A DADOS

Evite hardcode.

Em vez de:

```
if (unitType == 5)
{
    damage = 20;
}
```

prefira:

```
{
  "id": 5,
  "name": "Archer",
  "damage": 20
}
```

Unidades, prédios, tecnologias e missões devem ser definidas por dados sempre que possível.

---

41. MILESTONES DO REMAKE

Milestone 0 — Ferramentas

- inventário;
- parsers;
- conversores;
- viewers;
- análise de arquivos.

Milestone 1 — Janela e renderização

- janela;
- fullscreen;
- renderer;
- sprites;
- tiles.

Milestone 2 — Mapa

- carregar;
- desenhar;
- câmera;
- colisão;
- terreno.

Milestone 3 — Unidades

- spawn;
- seleção;
- animação;
- movimento.

Milestone 4 — Pathfinding

- destino;
- obstáculos;
- grupos.

Milestone 5 — Economia

- recursos;
- coleta;
- transporte;
- depósito;
- armazenamento.

Milestone 6 — Construções

- placement;
- construção;
- footprint;
- produção.

Milestone 7 — Combate

- ataque;
- dano;
- projéteis;
- morte.

Milestone 8 — Pesquisa

- tecnologias;
- upgrades.

Milestone 9 — IA

- coleta;
- economia;
- construção;
- defesa;
- ataque.

Milestone 10 — Eventos

- triggers;
- timers;
- spawn;
- condições.

Milestone 11 — Missões

- campanha;
- objetivos;
- vitória;
- derrota.

Milestone 12 — Save/Load

- save moderno;
- load;
- importação opcional do save original.

Milestone 13 — UI

- HUD;
- minimapa;
- menus;
- tooltips.

Milestone 14 — Áudio

- música;
- efeitos;
- vozes.

Milestone 15 — Polimento

- bugs;
- compatibilidade;
- desempenho;
- resolução.

---

42. VERTICAL SLICE

Antes de tentar recriar o jogo inteiro, produza uma versão mínima contendo:

- mapa real;
- uma unidade;
- seleção;
- movimentação;
- coleta;
- depósito;
- uma construção;
- um inimigo;
- combate;
- um evento;
- save/load.

Essa versão deve usar dados extraídos do jogo original.

---

43. TESTES A/B

Sempre que possível, compare:

```
Original
vs
Remake
```

Compare:

- velocidade;
- posição;
- coleta;
- dano;
- produção;
- research;
- animação;

- IA;
- eventos;
- timing.

Registre diferenças.

---

44. TESTES AUTOMATIZADOS

Crie testes para:

- parsing;
- resources;
- units;
- buildings;
- movement;
- pathfinding;
- combat;
- research;
- events;
- save/load.

---

45. FIDELIDADE

Sempre que houver dúvida entre:

```
melhorar o comportamento
```

e:

```
reproduzir o comportamento original
```

priorize primeiro a reprodução fiel.

Melhorias podem ser adicionadas posteriormente como opções.

---

46. IMPORTAÇÃO DOS ASSETS ORIGINAIS

Sempre que possível, faça o remake carregar os arquivos originais do usuário.

Fluxo ideal:

```
RisingLandsRemake.exe
    ↓
detecta instalação/CD original
    ↓
importa assets
    ↓
converte
    ↓
gera cache
    ↓
executa
```

Isso evita depender de redistribuir conteúdo original.

---

47. FERRAMENTAS QUE PODEM SER CRIADAS

Crie ferramentas específicas quando necessário:

```
FileInspector
SaveParser
SaveEditor
```

```
MapViewer
MapConverter
SpriteExtractor
SpriteViewer
SoundExtractor
EventDecompiler
MissionViewer
ExeDiff
MemoryInspector
DataExporter
```

Cada ferramenta deve ter documentação.

---

48. FORMATO DE ENTREGA

A análise deve resultar em uma documentação navegável:

```
documentation/
├── overview.md
├── architecture_overview.md
├── file_inventory.md
├── executable_map.md
├── exe_diff.md
├── strings.md
├── game_loop.md
├── timing.md
├── maps.md
├── terrain.md
├── units.md
├── unit_states.md
├── buildings.md
├── resources.md
├── economy.md
├── research.md
├── combat.md
├── ai.md
├── pathfinding.md
├── events.md
├── spawn_system.md
├── missions/
├── graphics.md
├── animations.md
├── audio.md
├── ui.md
├── input.md
├── save_format.md
├── memory_map.md
├── crashes.md
├── research_log.md
├── game_specification.md
└── remake_plan.md
```

---

49. RELATÓRIO DE PROGRESSO

Ao final de cada sessão de análise, produza:

```
reports/progress_YYYY-MM-DD.md
```

Incluindo:

- arquivos analisados;
- descobertas;
- hipóteses;
- bugs;

- formatos identificados;
- formatos pendentes;
- ferramentas criadas;
- próximos passos.

---

50. CONTROLE DE DESCOBERTAS

Para toda conclusão importante, use:

```
Status:
CONFIRMADO / FORTE EVIDÊNCIA / PROVÁVEL / HIPÓTESE / DESCONHECIDO

Fonte:
arquivo / offset / função / experimento

Evidência:

Conclusão:

Próximo passo:
```

---

51. NÃO RECOMEÇAR ANÁLISES

Antes de iniciar qualquer investigação, consulte:

```
documentation/research_log.md
```

Não repita trabalho já realizado.

Atualize a documentação em vez de criar descobertas desconectadas.

---

52. NÃO PARE NO PRIMEIRO RESULTADO

Se encontrar:

```
"isso parece ser um mapa"
```

não pare.

Descubra:

- cabeçalho;
- dimensões;
- layout;
- registros;
- referências;
- dados;
- compressão;
- relação com outros arquivos.

Transforme hipótese em especificação sempre que possível.

---

53. RELAÇÃO ENTRE ARQUIVOS

Crie um grafo de dependências.

Exemplo:

```
Mission
 ├─ Map
 ├─ Events
 ├─ Units
```

```
├─ AI
├─ Text
└─ Audio
```

Crie:

```
documentation/data_relationships.md
```

---

54. RELAÇÃO ENTRE DISCO E MEMÓRIA

Sempre que possível, identifique:

```
arquivo no disco
↓
loader
↓
estrutura na memória
↓
comportamento
```

Isso é especialmente importante para:

- mapas;
- unidades;
- recursos;
- eventos;
- IA;
- saves.

---

55. PSEUDOCÓDIGO

Para funções importantes do EXE, gere pseudocódigo.

Exemplo:

```
void UpdateUnit(Unit* unit)
{
    switch(unit->state)
    {
        case STATE_IDLE:
            ...
            break;

        case STATE_MOVE:
            ...
            break;
    }
}
```

Não invente nomes definitivos sem evidência.

Use nomes provisórios como:

```
FUN_00412340
Possible_UpdateUnit
```

até confirmação.

---

56. NOMENCLATURA

Quando uma função for identificada:

```
FUN_0041A700
```

pode evoluir para:

```
Unit_GetTarget
```

Mas registre a mudança no research log.

---

57. FORMATO DE DADOS MODERNO

Crie uma camada intermediária em JSON/YAML/binário moderno para dados identificados.

Exemplo:

```
Data/
├── units.json
├── buildings.json
├── technologies.json
├── missions/
├── maps/
└── resources.json
```

---

58. COMPATIBILIDADE COM WINDOWS 10/11

O remake não deve depender de:

- DirectX 3;
- DirectDraw antigo;
- DirectPlay antigo;
- DLLs obsoletas;
- comportamento dependente de CPU.

Deve usar APIs modernas.

---

59. RESOLUÇÕES

O remake deve suportar inicialmente:

```
800x600
```

para reproduzir a interface original.

Depois permitir:

```
1280x720
1920x1080
2560x1440
4K
```

sem alterar a lógica do jogo.

---

60. TIMING DETERMINÍSTICO

A lógica do jogo deve ser desacoplada do FPS.

Exemplo:

```
simulation tick = fixo
render FPS = independente
```

Isso evita os problemas de velocidade observados no executável original.

---

61. SAVE MODERNO

O remake deve usar um formato robusto e versionado.

Exemplo:

```
{
  "version": 1,
  "mission": 2,
  "gameTime": 1234,
  "resources": {},
  "units": [],
  "buildings": [],
  "events": []
}
```

---

62. IMPORTADOR DE SAVE ANTIGO

Se possível, crie:

```
OriginalSaveImporter
```

para importar `.sav` antigos.

---

63. DOCUMENTAÇÃO DE BUGS DO ORIGINAL

Registre também bugs descobertos no jogo original.

Exemplo:

```
Bug:
uso de índice FFFF como objeto válido

Resultado:
access violation

Comportamento esperado:
ignorar objeto inexistente
```

Crie:

```
documentation/original_bugs.md
```

---

64. PRESERVAÇÃO

O objetivo também é preservar conhecimento sobre o jogo.

A documentação deve ser suficiente para outra pessoa continuar o projeto sem depender da memória da IA atual.

---

65. ORDEM DE EXECUÇÃO INICIAL

Comece exatamente nesta ordem:

11. criar estrutura de trabalho;
12. criar hashes dos originais;
13. inventariar todos os arquivos;
14. identificar formatos conhecidos;
15. extrair strings;
16. analisar arquivos de texto/configuração;
17. analisar EXE;
18. comparar builds;
19. mapear referências a arquivos;
20. identificar unidades;
21. identificar construções;
22. identificar recursos;

23. identificar mapas;
24. identificar eventos;
25. identificar gráficos;
26. identificar áudio;
27. analisar saves;
28. documentar game loop;
29. documentar timing;
30. criar especificação inicial;
31. criar primeiro parser;
32. criar primeiro viewer;
33. criar primeiro protótipo do remake.

---

66. PRIMEIRA ENTREGA OBRIGATÓRIA

Antes de começar a desenvolver o remake completo, entregue:

```
1. Inventário de arquivos
2. Arquitetura inicial
3. Lista de formatos
4. Mapa dos executáveis
5. Mapa dos dados
6. Lista de sistemas conhecidos
7. Lista de sistemas ainda desconhecidos
8. Prioridades de engenharia reversa
9. Riscos
10. Roadmap do remake
```

---

67. SEGUNDA ENTREGA OBRIGATÓRIA

Crie um protótipo capaz de:

- abrir uma janela;
- carregar um mapa real;
- exibir terreno real;
- carregar pelo menos um sprite;
- mover câmera;
- selecionar uma unidade;
- movimentar uma unidade.

---

68. TERCEIRA ENTREGA OBRIGATÓRIA

Crie uma vertical slice contendo:

- mapa;
- trabalhadores;
- coleta;
- recursos;
- depósito;
- construção;
- inimigo;
- combate;
- evento;
- vitória/derrota;
- save/load.

---

69. RESULTADO FINAL ESPERADO

Ao final do projeto devemos possuir:

34. compreensão da arquitetura do Rising Lands;
35. formatos de dados documentados;
36. mapas decodificados;
37. sprites extraídos;
38. sons extraídos;
39. unidades documentadas;
40. construções documentadas;
41. tecnologias documentadas;
42. economia documentada;
43. combate documentado;
44. IA documentada;
45. eventos documentados;
46. missões documentadas;
47. saves documentados;
48. timing documentado;
49. memória documentada quando necessário;
50. bugs conhecidos documentados;
51. ferramentas de conversão;
52. parsers;
53. viewers;
54. especificação completa do jogo;
55. roadmap técnico;
56. engine moderna;
57. primeira missão jogável;
58. campanha progressivamente recriada;
59. versão compatível com Windows 10/11.

---

70. OBJETIVO FINAL

O objetivo não é apenas:

> *"fazer o Rising Lands abrir em Windows moderno".*

O objetivo é produzir uma nova implementação moderna, compreensível, documentada e sustentável do Rising Lands, capaz de:

- rodar em Windows 10;
- rodar em Windows 11;
- funcionar em hardware moderno;
- eliminar dependências antigas;
- eliminar bugs de compatibilidade;
- manter timing correto;
- preservar gameplay;
- preservar campanhas;
- preservar IA;
- preservar eventos;
- preservar mapas;
- preservar assets;
- permitir manutenção;
- permitir melhorias opcionais;
- preservar o jogo para o futuro.

Trate o projeto como uma combinação de:

```
preservação digital
+
```

```
engenharia reversa
+
documentação técnica
+
reimplementação de engine
+
remake fiel
```

Não pule etapas críticas.

Não esconda incertezas.

Não invente dados.

Documente tudo.

Trabalhe de forma incremental, verificável e reproduzível.